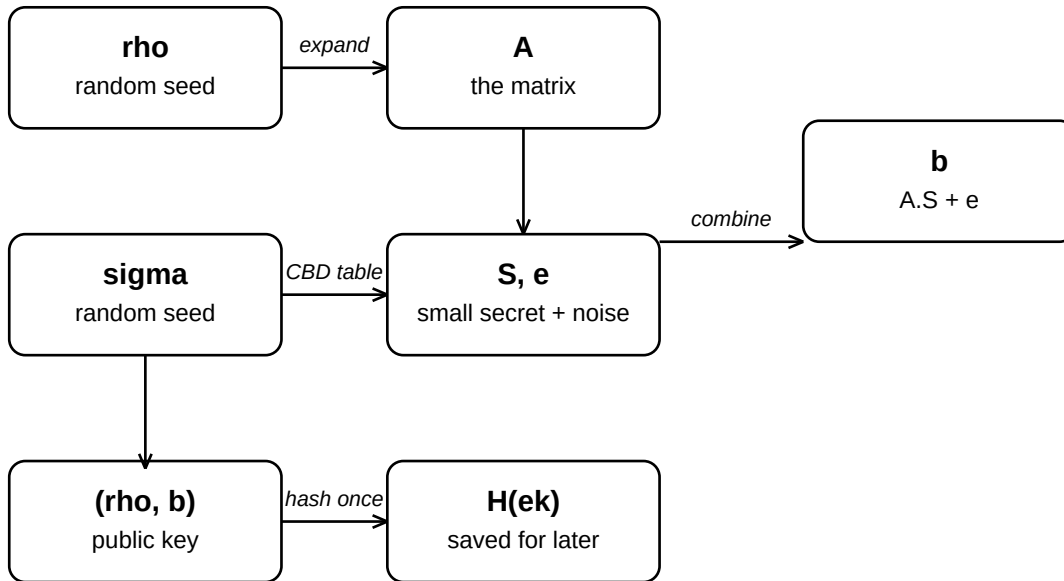


Act 1: Bob's KeyGen



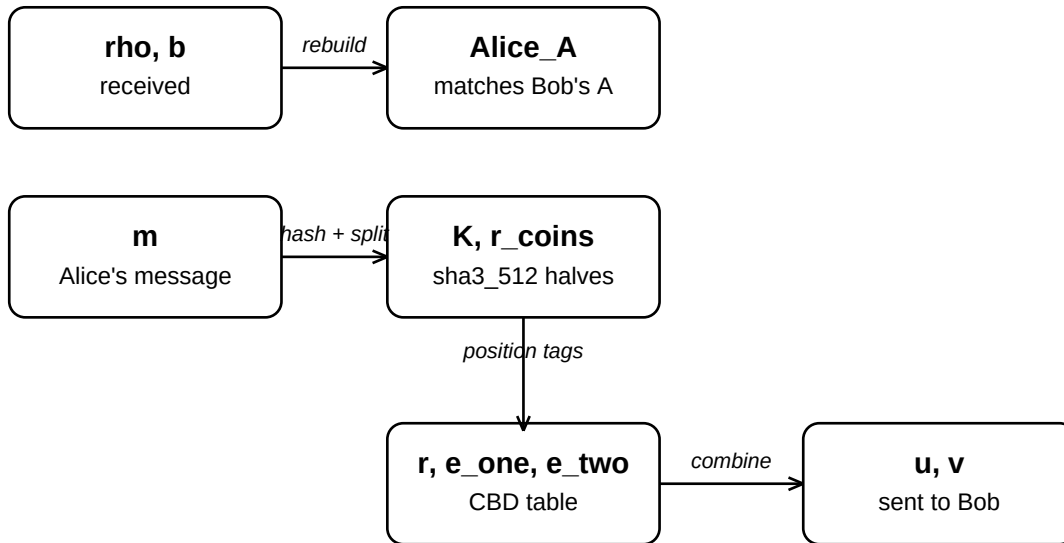
Bob makes a small random seed (**rho**) and expands it into the full matrix **A** - he never has to send the whole matrix, just this one small seed.

A second seed (**sigma**) gets tagged with a different position for each polynomial and turned into small values via the CBD table - this gives **S** (Bob's private secret) and **e** (noise).

$b = A.S + e$ is computed with normal `poly_add/poly_mult`.

Bob's public key is just **(rho, b)** - never the whole matrix. He also hashes his own public key once, right now, and saves the result (**H(ek)**) so he never has to redo that specific hash later.

Act 2: Alice's Encapsulate



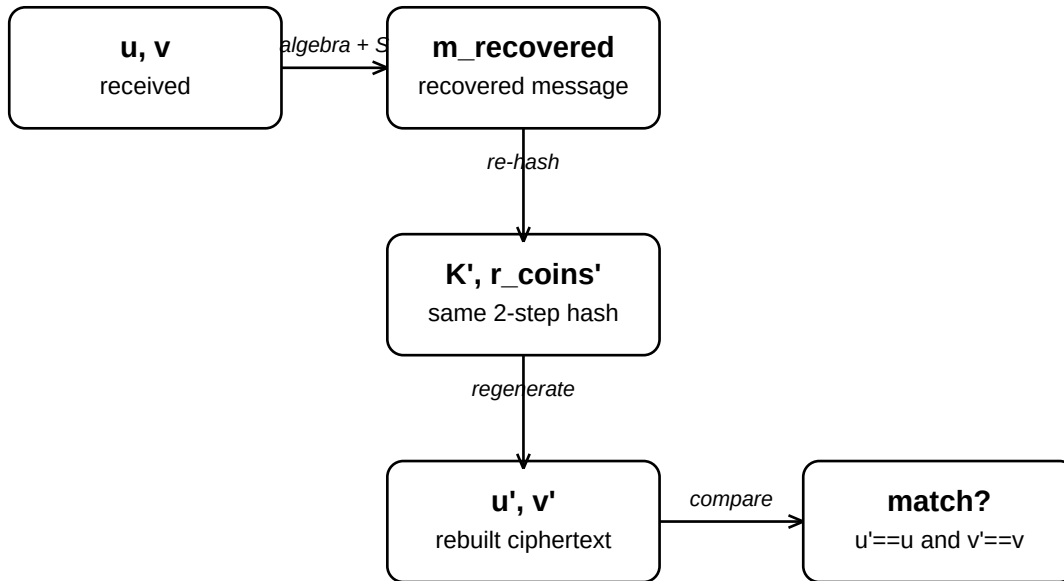
Alice rebuilds A herself from the rho she received - same expansion Bob used. She also computes $H(ek)$ herself, from (rho, b).

Alice picks her message m (real randomness, via secrets). She hashes $(m + H(ek))$ with sha3_512, getting 64 bytes back, and splits that in half: the first half becomes K (the real shared secret, saved for later), the second half becomes r_coins.

r_coins gets tagged with a position for each polynomial needed and run through the CBD table, giving r, e_one, e_two - the noise for this encryption, derived from m instead of picked independently.

u and v get built with the normal formulas, then sent to Bob.

Act 3: Bob's Decapsulate



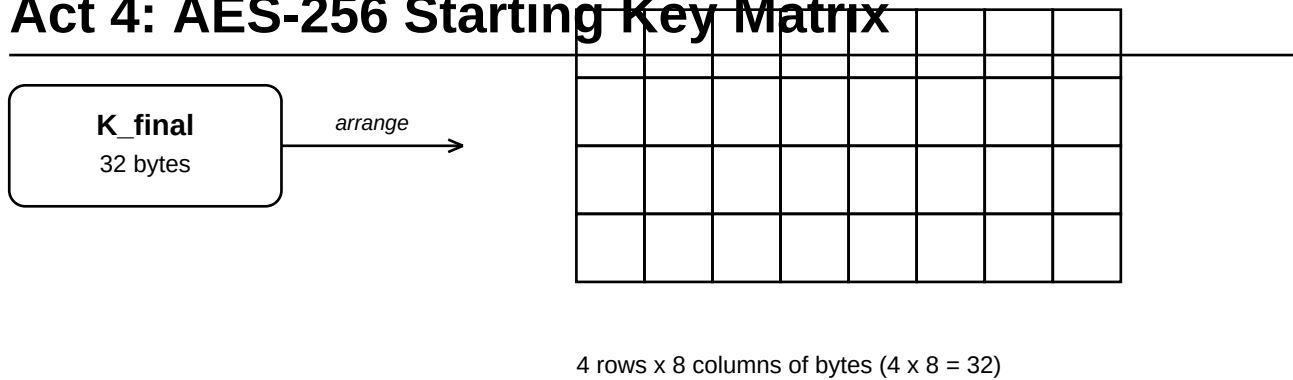
Bob uses his private S against the (u, v) he received to recover $m_recovered$ - pure algebra, same as the very first toy build.

Bob re-hashes $(m_recovered + H(ek))$ - reusing his already-stored $H(ek)$ instead of recomputing it - and gets K' and r_coins' the same way Alice did.

r_coins' regenerates r' , e_one' , e_two' , which rebuild what u and v should have looked like, using Bob's own A and b .

If the rebuilt (u', v') matches the real (u, v) Alice sent, K' is confirmed as the genuine shared secret - and it matches Alice's K exactly. (The fallback for a mismatch isn't built yet.)

Act 4: AES-256 Starting Key Matrix



K_final is 32 bytes - the real, confirmed shared secret from Act 3.

AES-256 needs its key arranged into a grid before the key schedule can expand it: 4 rows by 8 columns of bytes (4 x 8 = 32, exactly the size of K_final).

Bytes fill DOWN each column first, then move to the next column - AES's standard fill order. So byte 0 goes to row 0/col 0, byte 1 to row 1/col 0, byte 2 to row 2/col 0, byte 3 to row 3/col 0, byte 4 starts the next column at row 0/col 1, and so on.

This 4x8 grid is where AES-256's key schedule (turning one key into 15 separate round keys) would start - not built yet, next piece.